

Authored by Ryan Walker Brown, Case Western Reserve University. All use of this in academic spaces should cite upcoming paper "Simulating Robotic Locomotion in Sand: Resistive Force Theory in an Open-Source Physics Engine"

```
%reset -f
from sim_fxn_lib import *
import csv
from matplotlib import cm
import cv2
import shutil
```

```
xml_path = 'Robot_Sand.xml'
typ = "example"
RFTCOEFF = 3.75
save_every = 8
repeats = 5
tMax = 5
dt = 0.001
model, data, renderer, t, dt, frames, framerate, sand_h_id, stl_path = initial
    tMax,
    dt,
    xml_path,
    'sandflipper.stl',
    camera_name="plate_camera"
)
```

```
numSteps = len(t)
tMax = t[-1]
pos_actuator_ids = {}
actuator_names = [
    "mid segment front right_p", "distal segment front right_p",
    "mid segment middle right_p", "distal segment middle right_p",
    "mid segment back right_p", "distal segment back right_p",
    "mid segment front left_p", "distal segment front left_p",
    "mid segment middle left_p", "distal segment middle left_p",
    "mid segment back left_p", "distal segment back left_p",
]

for name in actuator_names:
    pos_actuator_ids[name] = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTU

control_pos = load_control_data_yaml('Example_Gait.yaml')
new_len = int(numSteps)
```

```

control_pos_mid_fr = interpolate_array(control_pos['theta1_R1'], new_len, repeat)
control_pos_distal_fr = interpolate_array(control_pos['theta2_R1'], new_len, repeat)
control_pos_mid_mr = interpolate_array(control_pos['theta1_R2'], new_len, repeat)
control_pos_distal_mr = interpolate_array(control_pos['theta2_R2'], new_len, repeat)
control_pos_mid_br = interpolate_array(control_pos['theta1_R3'], new_len, repeat)
control_pos_distal_br = interpolate_array(control_pos['theta2_R3'], new_len, repeat)
control_pos_mid_fl = interpolate_array(control_pos['theta1_L1'], new_len, repeat)
control_pos_distal_fl = interpolate_array(control_pos['theta2_L1'], new_len, repeat)
control_pos_mid_ml = interpolate_array(control_pos['theta1_L2'], new_len, repeat)
control_pos_distal_ml = interpolate_array(control_pos['theta2_L2'], new_len, repeat)
control_pos_mid_bl = interpolate_array(control_pos['theta1_L3'], new_len, repeat)
control_pos_distal_bl = interpolate_array(control_pos['theta2_L3'], new_len, repeat)

sand_h_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "sand_height")

dactyl_sinkage = {name: [] for name in [
    "prox_fr_dactyl_tip",
    "prox_mr_dactyl_tip",
    "prox_br_dactyl_tip",
    "prox_fl_dactyl_tip",
    "prox_ml_dactyl_tip",
    "prox_bl_dactyl_tip"
]}

dactyl_site_ids = {name: mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, name)
                    for name in dactyl_sinkage.keys()}

entities = get_named_bodies_from_xml(xml_path)

body, vertices, faces, mesh = load_and_process_mesh(stl_path, scale_factor=100)
num_sites = model.nsite - 7
num_faces = len(mesh.triangles)
num_active_sites = min(num_sites, num_faces)

vertices_dict = {}
faces_dict = {}
body_dict = {}
mesh_dict = {}

for body_name in entities:
    stl_path = f"asset\\{body_name}.stl"
    body, vertices, faces, mesh = load_and_process_mesh(stl_path, scale_factor=100)
    vertices_dict[body_name] = vertices
    faces_dict[body_name] = faces
    body_dict[body_name] = body
    mesh_dict[body_name] = mesh

    body_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, body_name)

def initialize_sites_for_all_bodies(model, data, body_dict, sitename="force"):
    for body_name, mesh in mesh_dict.items():
        initialize_sites_on_mesh(

```

```

        model=model,
        data=data,
        mesh=mesh,
        sitename=f"{sitename}_{body_name}",
        bodyname=body_name
    )

initialize_sites_for_all_bodies(model, data, body_dict, sitename="force")

# def generate_site_lines(body_names, sites_per_body=500, output_path="sites.txt")
#     with open(output_path, "w") as f:
#         for body_name in body_names:
#             for i in range(sites_per_body):
#                 line = f'<site name="force_{body_name}_site_{i}" pos="0 0 0.
#                 f.write(line)
#     # print(f"Site definitions written to {output_path}")

body_list = entities
# generate_site_lines(body_list, sites_per_body=500, output_path="sites.txt")

v = {}
for entity in entities:
    v[f'fm_{entity}'] = []
    v[f'mm_{entity}'] = []

motion_data = {
    "time": [],
    "x": [],
    "y": [],
    "z": [],
    "roll": [],
    "pitch": [],
    "yaw": []
}

camera_list = ["diag"]
frames = {cam: [] for cam in camera_list}
plate_pos = []

F_full_sorted_prev = np.zeros_like(faces)
prev_body_pos_dict = {name: None for name in entities}
prev_body_quat_dict = {name: None for name in entities}

applied_force = True
once_submerged = False
frame_dir = "frames"
os.makedirs(frame_dir, exist_ok=True)
for filename in os.listdir(frame_dir):
    file_path = os.path.join(frame_dir, filename)
    try:

```

```

        if os.path.isfile(tile_path) or os.path.islink(tile_path):
            os.unlink(file_path)
        elif os.path.isdir(file_path):
            shutil.rmtree(file_path)
    except Exception as e:
        print('Failed to delete %s. Reason: %s' % (file_path, e))

# MAIN SIM
body_velocities = {body_name: [] for body_name in entities}
body_angular_velocities = {body_name: [] for body_name in entities}
site_ids = {}
SUB = False
for body_name in body_list:
    site_ids[f"{body_name}"] = [
        mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, f"force_{body_name}")
        for s in range(num_sites)
    ]
site_id_arrays = {body_name: np.array(ids) for body_name, ids in site_ids.items()}
face_sort_order = {}
sorted_faces_cache = {}
sorted_site_ids_cache = {}

for body_name in body_list:
    faces, verts = faces_dict[body_name], vertices_dict[body_name]
    centroids_x = np.mean(verts[faces], axis=1)[: , 0]
    order = np.argsort(centroids_x)
    face_sort_order[body_name] = order
    sorted_faces_cache[body_name] = faces[order]
    sorted_site_ids_cache[body_name] = np.array(site_ids[body_name])[order]

face_areas_cache = {
    body_name: calculate_face_areas(vertices, sorted_faces_cache[body_name])
    for body_name in body_list
}

```

```

for i in range(len(t)):

```

```

    data.ctrl[pos_actuator_ids["distal segment front right_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["mid segment front right_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["distal segment middle right_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["mid segment middle right_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["distal segment back right_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["mid segment back right_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["distal segment front left_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["mid segment front left_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["distal segment middle left_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["mid segment middle left_p"]] = control_pos_m
    data.ctrl[pos_actuator_ids["distal segment back left_p"]] = control_pos_m

```

```

data.ctrl[pos_actuator_ids["mid segment back left_p"]] = control_pos_mid

mujoco.mj_step(model, data)
data.qfrc_applied[:] = 0

global_pos_sand = data.site_xpos[sand_h_id]

for body_name in body_list:

    body_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, body_name)
    if body_name == "plate":
        plate_pos.append(np.array(data.xpos[body_id]))

    ids_arr = site_id_arrays[body_name]
    site_z = data.site_xpos[ids_arr, 2]
    SUB = bool(np.any(site_z < global_pos_sand[2]))

    body_pos = np.array(data.xpos[body_id])
    quat_now = np.array(data.xquat[body_id])
    if i > 0 and prev_body_pos_dict[body_name] is not None:
        prev_body_pos = prev_body_pos_dict[body_name]
        prev_body_quat = prev_body_quat_dict[body_name]
        body_linear_velocity = (body_pos - prev_body_pos) / dt
        quat_prev = np.array(prev_body_quat)
        r_now = scipy.spatial.transform.Rotation.from_quat([quat_now[1]
        r_prev = scipy.spatial.transform.Rotation.from_quat([quat_prev[1]
        rotvec = (r_now * r_prev.inv()).as_rotvec()
        body_angular_velocity = rotvec / dt
    else:
        body_linear_velocity = np.zeros(3)
        body_angular_velocity = np.zeros(3)
    body_velocities[body_name].append(body_linear_velocity.copy())
    body_angular_velocities[body_name].append(body_angular_velocity.copy())
    prev_body_pos_dict[body_name] = body_pos.copy()
    prev_body_quat_dict[body_name] = quat_now.copy()

    body_orientation_world = data.xquat[body_id]
    euler_angles = quaternion_to_euler(body_orientation_world)
    roll, pitch, yaw = euler_angles

    F_muj, M_muj = np.array([0, 0, 0]), np.array([0, 0, 0])

    if SUB:
        if once_submerged is False:
            print(f"Body {body_name} submerged at time {t[i]:.3f}s with
            once_submerged = True

    F mui. M mui. Fi mat. Mi mat. include. F full. RFTCOFF = rft 3D

```

```

        body_dict[body_name],
        body_pos,
        [yaw, pitch, roll],
        body_linear_velocity,
        body_angular_velocity,
        RFTCOEFF=RFTCOEFF,
        sand_height_m=global_pos_sand[2]
    )

    order          = face_sort_order[body_name]
    faces_sorted    = sorted_faces_cache[body_name]
    site_ids_sorted = sorted_site_ids_cache[body_name]
    F_full_sorted   = F_full[order]
    face_areas      = face_areas_cache[body_name]
    stress_plot     = F_full_sorted / face_areas[:, np.newaxis]
    stress_magnitude = np.linalg.norm(stress_plot, axis=1)
    if stress_magnitude.size > 0 and stress_magnitude.max() != stress_magnitude.min():
        norm_stress = (stress_magnitude - stress_magnitude.min()) /
    else:
        norm_stress = np.zeros_like(stress_magnitude)
    face_colors = cm.Blues(norm_stress)[: , :3]

    if "prev_forces" not in globals():
        prev_forces = {}

    distal_bodies = {
        "distal segment front right", "distal segment front left",
        "distal segment middle right", "distal segment middle left",
        "distal segment back right", "distal segment back left"
    }

    for face_idx, site_id in enumerate(site_ids_sorted):
        if face_idx < len(face_colors):
            if body_name in distal_bodies:
                color = np.concatenate([face_colors[face_idx], [.8]])
            else:
                color = np.array([0.5, 0.5, 0.5, 0], dtype=np.float32)
        else:
            color = np.array([0.5, 0.5, 0.5, 0], dtype=np.float32)
        model.site_rgba[site_id, :] = color

        if face_idx < len(F_full_sorted) and applied_force is True:
            force_applied_at_site = np.array(-F_full_sorted[face_idx]
            alpha = .5
            if site_id in prev_forces:
                force_applied_at_site = (
                    alpha * force_applied_at_site +
                    (1 - alpha) * prev_forces[site_id]
                )
            prev_forces[site_id] = force_applied_at_site

```

```

        force_applied_at_site = force_applied_at_site.reshape((3
mujoco.mj_applyFT(
            model, data,
            force_applied_at_site,
            np.zeros((3, 1), dtype=np.float64),
            np.array(data.site_xpos[site_id]).reshape((3, 1)),
            body_id,
            data.qfrc_applied
        )

        v[f'fm_{body_name}'].append(F_muj)
        v[f'mm_{body_name}'].append(M_muj)
    else:
        v[f'fm_{body_name}'].append(np.array([0, 0, 0]))
        v[f'mm_{body_name}'].append(np.array([0, 0, 0]))
    for dactyl_name, site_id in dactyl_site_ids.items():
        tip_pos = data.site_xpos[site_id]
        sinkage = global_pos_sand[2] - tip_pos[2]
        dactyl_sinkage[dactyl_name].append(sinkage)

if i % save_every == 0:

    renderer.update_scene(data, camera="diag")
    frame = renderer.render()
    cv2.imwrite(f"frames/{typ}_RFT_{RFTCOEFF:.2f}_frame_{i:04d}.png", fr
    cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    print(f"wrote frame {typ}_RFT_{RFTCOEFF:.2f}_frame_{i:04d}.png, RFT

```

```

with open(f"W{typ}_plate_position{RFTCOEFF}.csv", "w", newline="") as f:

```

```

    writer = csv.writer(f)
    writer.writerow(["X [m]", "Y [m]", "Z [m]"]) # header
    for i in range(len(plate_pos)):
        row = [f"{coord:.4f}" for coord in plate_pos[i]]
        # print(row)
        writer.writerow(row)

```

```

def create_video_from_frames(typ, frame_folder, frame_prefix, save_every, rf
    fps = 1000 / save_every # Match the simulation timestep
    if output_name is None:
        output_video = f"{typ}_{rft_coeff}.mp4"
    else:
        output_video = output_name
    frames = sorted([f for f in os.listdir(frame_folder)
                     if f.startswith(frame_prefix) and f.endswith(".png")])
    first_frame = cv2.imread(os.path.join(frame_folder, frames[0]))
    height, width, _ = first_frame.shape
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')

```

```
out = cv2.VideoWriter(output_video, fourcc, fps, (width, height))
# Write all frames to video
for f in frames:
    img = cv2.imread(os.path.join(frame_folder, f))
    if img.ndim == 2:
        img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    out.write(img)
out.release()
print(f"Video saved to {output_video}")
return output_video
```

```
create_video_from_frames(
    typ=typ,
    frame_folder=r"frames",
    frame_prefix=f"{typ}_RFT_3.75_frame_",
    save_every=save_every,
    rft_coeff=RFTCOEFF
)
```